S.B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT & RESEARCH, NAGPUR

Practical 07

Aim: Develop a program to manage resource allocation for five processes (Google Drive, Firefox, Word Processor, Excel, and PowerPoint) using four types of resources (Printer, ROM, Hard Disk, and RAM). The program takes input for allocated, maximum, and available resources, calculates the current need of each process, and determines if a safe execution order exists using the Banker's Algorithm.

Name : Atul N. Patle

USN : CM25D003

Sem/Year : IV/II

Academic Session : 2025-2026

Date of Performance : 10/03/2026

Date of Submission : /03/2026

- **Aim:** Develop a program to manage resource allocation for five processes (Google Drive, Firefox, Word Processor, Excel, and PowerPoint) using four types of resources (Printer, ROM, Hard Disk, and RAM). The program takes input for allocated, maximum, and available resources, calculates the current need of each process, and determines if a safe execution order exists using the Banker's Algorithm.

- **Objectives:**
  1. Implement the Banker's Algorithm to manage resource allocation and prevent deadlocks.
  2. Calculate the current resource need for each process based on allocated and maximum resources.
  3. Determine a safe execution sequence for processes if resources are available.

- **Requirements:**
  - ✓ **Hardware Requirements:**
    - Processor: Minimum 1 GHz
    - RAM: 512 MB or higher
    - Storage: 100 MB free space
  - ✓ **Software Requirements:**
    - Operating System: Linux/Unix-based
    - Shell: Bash 4.0 or higher
    - Text Editor: Nano, Vim, or any preferred editor

- **Theory:**

Banker's Algorithm in Operating System

Banker's Algorithm is a resource allocation and deadlock avoidance algorithm used in operating systems. It ensures that a system remains in a safe state by carefully allocating resources to processes while avoiding unsafe states that could lead to deadlocks.

- The Banker's Algorithm is a smart way for computer systems to manage how programs use resources, like memory or CPU time.
- It helps prevent situations where programs get stuck and can not finish their tasks. This condition is known as deadlock.
- By keeping track of what resources each program needs and what's available, the banker algorithm makes sure that programs only get what they need in a safe order.

Why Banker's Algorithm is Named So?

The banker's algorithm is named so because it is used in the banking system to check whether a loan can be sanctioned to a person or not. Suppose there are n number of account holders in a bank and the total sum of their money is S. Let us assume that the bank has a certain amount of money Y. If a person applies for a loan then the bank first subtracts the loan amount from the total money that the bank has (Y) and if the remaining amount is greater than S then only the loan is sanctioned. It is done because if all the account holders come to withdraw their money then the bank can easily do it.

It also helps the OS to successfully share the resources between all the processes. The banker's algorithm uses the notation of a safe allocation state to ensure that granting a resource request cannot lead to a deadlock either immediately or in the future. In other words, the bank would never allocate its money in such a way that it can no longer satisfy the needs of all its customers. The bank would try to be in a safe state always.

Components of the Banker's Algorithm

The following **Data structures** are used to implement the Banker's Algorithm:

Let **'n'** be the number of processes in the system and **'m'** be the number of resource types.

1. **Available**
   - It is a 1-D array of size **'m'** indicating the number of available resources of each type.
   - Available[j] = k means there are **'k'** instances of resource type **Rj**
2. **Max**
   - It is a 2-d array of size **'n*m'** that defines the maximum demand of each process in a system.
   - Max[i, j] = k means process **Pi** may request at most **'k'** instances of resource type **Rj.**
3. **Allocation**
   - It is a 2-d array of size **'n*m'** that defines the number of resources of each type currently allocated to each process.
   - Allocation[i, j] = k means process **Pi** is currently allocated **'k'** instances of resource type **Rj**
4. **Need**
   - It is a 2-d array of size **'n*m'** that indicates the remaining resource need of each process.
   - Need [i, j] = k means process **Pi** currently needs **'k'** instances of resource type **Rj**
   - Need [i, j] = Max [i, j] – Allocation [i, j]

Allocation specifies the resources currently allocated to process Pi and Need specifies the additional resources that process Pi may still request to complete its task.

Banker's algorithm consists of a Safety algorithm and a Resource request algorithm.

Banker's Algorithm

1. Active := *Running U Blocked*
for k = 1 to r:
New_request[k] := Requested_resources[requesting_process, k]
2. Simulated_allocation := *Allocated_resources*
for k = 1 to r: // Compute projected allocation state
Simulated_allocation[requesting_process, k] :=
Simulated_allocation[requesting_process, k] + New_request[k]
3. Feasible := *true*
for k = 1 to r: // Check if projected allocation state is feasible
if Total_resources[k] < Simulated_total_alloc[k]:
Feasible := false
4. if Feasible = *true:*
while set Active contains a process P1 such that:
for all k, Total_resources[k] – Simulated_total_alloc[k] >=
Max_need[P1, k] – Simulated_allocation[P1, k]:
Delete P1 from Active
for k = 1 to r:
Simulated_total_alloc[k] :=
Simulated_total_alloc[k] – Simulated_allocation[P1, k]
5. if set Active is empty:
for k = 1 to r: // Delete the request from pending requests
Requested_resources[requesting_process, k] := 0
for k = 1 to r: // Grant the request
Allocated_resources[requesting_process, k] :=
Allocated_resources[requesting_process, k] + New_request[k]
Total_alloc[k] := Total_alloc[k] + New_request[k]

Safety Algorithm

The algorithm for finding out whether or not a system is in a safe state can be described as follows:

1. Let Work and Finish be vectors of length 'm' and 'n' respectively.
   Initialize: Work = Available
   Finish[i] = false; for i=1, 2, 3, 4….n
2. Find an i such that both
   2.1 Finish[i] = false
   2.2 Needi <= Work
   if no such i exists goto step (4)
3. Work = Work + Allocation[i]
   Finish[i] = true
   goto step (2)
4. if Finish [i] = true for all i then the system is in a safe state.

Resource-Request Algorithm

Let Requesti be the request array for process Pi. Requesti [j] = k means process Pi wants k instances of resource type Rj. When a request for resources is made by process Pi, the following actions are taken:

1. If Requesti <= Needi

Goto step (2) ; otherwise, raise an error condition, since the process has exceeded its maximum claim.

2. If Requesti <= Available

Goto step (3); otherwise, Pi must wait, since the resources are not available.

3. Have the system pretend to have allocated the requested resources to process Pi by modifying the state as

follows:

Available = Available – Requesti

Allocationi = Allocationi + Requesti

Needi = Needi– Requesti

Example: Considering a system with five processes P0 through P4 and three resources of type A, B, C. Resource type A has 10 instances, B has 5 instances and type C has 7 instances. Suppose at time t0 following snapshot of the system has been taken:

Q.1 What will be the content of the Need matrix?

Need [i, j] = Max [i, j] – Allocation [i, j]

So, the content of Need Matrix is:

Need Matrix Content:

The content of the **Need Matrix** is calculated using the following formula:

$$Need[i,j] = Max[i,j] - Allocation[i,j]$$

Where:

- **Max[i,j]** represents the maximum number of resources of type *j* that process *i* may require.
- **Allocation[i,j]** represents the number of resources of type *j* currently allocated to process *i*.
- **Need[i,j]** indicates the remaining resources required by process *i* to complete its execution.

The **Need Matrix** therefore shows how many additional resources each process needs to finish its task in the system.

Q.2 Is the system in a safe state? If Yes, then what is the safe sequence?
Applying the Safety algorithm on the given system,

Safety Algorithm (Banker's Algorithm)

1. Initialize **Work = Available** and **Finish[i] = false** for all processes.
2. Find a process **Pi** such that:
   - **Finish[i] = false**
   - **Need[i] ≤ Work**

3. If such a process is found, then:
   - **Work = Work + Allocation[i]**
   - **Finish[i] = true**
   - Add **Pi** to the safe sequence.

4. Repeat step 2 and 3 until all processes are marked **Finish = true**.

5. If all processes finish, the **system is in a safe state**; otherwise, it is **not safe**.

Result
After applying the safety algorithm, the **safe sequence** is:
P1 → P3 → P4 → P0 → P2

This means all processes can execute without causing **deadlock**, so the system is **safe**.

Q.3 What will happen if process P1 requests one additional instance of resource type A and two instances of resource type C?

Request by P1: (1, 0, 2)

1. First check **Request ≤ Need** → The request is valid because it does not exceed P1′s remaining need.

2. Check **Request ≤ Available** → The resources are available in the system.

3. Temporarily allocate the requested resources and apply the **Safety Algorithm**.

After applying the safety algorithm, the system **remains in a safe state** and all processes can still complete their execution.

Unsafe State in The Bankers Algorithm

In the context of the Banker's Algorithm, an **unsafe state** refers to a situation in which, all processes in a system currently hold resources within their maximum needs, there is no guarantee that the system can avoid a deadlock in the future. This state is contrasted with a **safe state**, where there exists a sequence of resource allocation and deallocation that guarantees that all processes can complete without leading to a deadlock.

Key Concepts in Banker's Algorithm

- **Safe State:** There exists at least one sequence of processes such that each process can obtain the needed resources, complete its execution, release its resources, and thus allow other processes to eventually complete without entering a deadlock.
- **Unsafe State:** Even though the system can still allocate resources to some processes, there is no guarantee that all processes can finish without potentially causing a deadlock.

Example of Unsafe State

Consider a system with three processes (P1, P2, P3) and 6 instances of a resource. Let's say:

- The available instances of the resource are: 1
- The current allocation of resources to processes is:
  - P1: 2
  - P2: 3
  - P3: 1
- The maximum demand (maximum resources each process may eventually request) is:
  - P1: 4
  - P2: 5
  - P3: 3

In this situation:

Need = Maximum Demand – Allocation

Process	Maximum Demand	Allocation	Need
P1	4	2	2
P2	5	3	2
P3	3	1	2

- P1 may need 2 more resources to complete.
- P2 may need 2 more resource to complete.
- P3 may need 2 more resources to complete.

However, there is only 1 resource available. Even though none of the processes are currently deadlocked, the system is in an unsafe state because there is no sequence of resource allocation that guarantees all processes can complete.

❖ CODE

```
#include <stdio.h>

int main()
{
    int n, m, i, j, k;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resource types: ");
    scanf("%d", &m);

    int alloc[n][m], max[n][m], need[n][m];
    int avail[m];
    int finish[n];
    int safe[n];

    printf("\nEnter Allocation Matrix:\n");
    for(i = 0; i < n; i++)
        for(j = 0; j < m; j++)
            scanf("%d", &alloc[i][j]);

    printf("\nEnter Maximum Matrix:\n");
    for(i = 0; i < n; i++)
        for(j = 0; j < m; j++)
            scanf("%d", &max[i][j]);

    printf("\nEnter Available Resources:\n");
    for(i = 0; i < m; i++)
        scanf("%d", &avail[i]);
```

```
printf("\n-----------------------------------\n");
printf("CALCULATING NEED MATRIX\n");
printf("-----------------------------------\n");

for(i = 0; i < n; i++)
    for(j = 0; j < m; j++)
        need[i][j] = max[i][j] - alloc[i][j];

printf("\nNeed Matrix:\n");
for(i = 0; i < n; i++)
{
    printf("P%d : ", i);
    for(j = 0; j < m; j++)
        printf("%d ", need[i][j]);
    printf("\n");
}

printf("\nInitial Available Resources: ");
for(i = 0; i < m; i++)
    printf("%d ", avail[i]);
printf("\n");

for(i = 0; i < n; i++)
    finish[i] = 0;

int count = 0;

printf("\n=====================================\n");
printf("STARTING BANKER'S ALGORITHM\n");
printf("=====================================\n");

while(count < n)
{
    int found = 0;

    for(i = 0; i < n; i++)
    {
        if(finish[i] == 0)
        {
            printf("\n-----------------------------------\n");
            printf("Checking Process P%d\n", i);
            printf("-----------------------------------\n");

            printf("Need of P%d : ", i);
            for(j = 0; j < m; j++)
                printf("%d ", need[i][j]);

            printf("\nAvailable   : ");
            for(j = 0; j < m; j++)
                printf("%d ", avail[j]);

            printf("\n");

            int possible = 1;

            for(j = 0; j < m; j++)
            {
```

```
            if(need[i][j] > avail[j])
                {
                    possible = 0;
                    printf("Resource %d not sufficient\n", j);
                    break;
                }
            }

            if(possible)
            {
                printf("All required resources available\n");
                printf("Process P%d CAN execute\n", i);

                printf("Releasing resources of P%d back to system\n", i);

                for(k = 0; k < m; k++)
                {
                    avail[k] += alloc[i][k];
                }

                printf("Updated Available Resources: ");
                for(k = 0; k < m; k++)
                    printf("%d ", avail[k]);

                printf("\n");

                safe[count] = i;
                count++;
                finish[i] = 1;
                found = 1;
            }
            else
            {
                printf("Process P%d CANNOT execute now\n", i);
            }
        }
    }

    if(found == 0)
    {
        printf("\nSYSTEM IS NOT IN SAFE STATE\n");
        return 0;
    }
}

printf("\n========================================\n");
printf("SYSTEM IS IN SAFE STATE\n");
printf("SAFE SEQUENCE:\n");

for(i = 0; i < n; i++)
    printf("P%d ", safe[i]);

printf("\n========================================\n");

return 0;
}
```

❖ **Output:**

```
Enter number of processes: 5
Enter number of resource types: 3

Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2

Enter Maximum Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3

Enter Available Resources:
3 3 2

-----------------------------------
CALCULATING NEED MATRIX
-----------------------------------

Need Matrix:
P0 : 7 4 3
P1 : 1 2 2
P2 : 6 0 0
P3 : 0 1 1
P4 : 4 3 1

Initial Available Resources: 3 3 2

=====================================
STARTING BANKER'S ALGORITHM
=====================================

---------------------------------
Checking Process P0
---------------------------------
Need of P0 : 7 4 3
Available   : 3 3 2
Resource 0 not sufficient

---------------------------------
Checking Process P1
---------------------------------
Need of P1 : 1 2 2
Available   : 3 3 2
All required resources available
Process P1 CAN execute
Releasing resources of P1 back to system
Updated Available Resources: 5 3 2
```

```
---------------------------------
Checking Process P2
---------------------------------
Need of P2 : 6 0 0
Available   : 5 3 2
Resource 0 not sufficient

---------------------------------
Checking Process P3
---------------------------------
Need of P3 : 0 1 1
Available   : 5 3 2
All required resources available
Process P3 CAN execute
Releasing resources of P3 back to system
Updated Available Resources: 7 4 3

---------------------------------
Checking Process P4
---------------------------------
Need of P4 : 4 3 1
Available   : 7 4 3
All required resources available
Process P4 CAN execute
Releasing resources of P4 back to system
Updated Available Resources: 7 4 5
Process P5 CANNOT execute now

---------------------------------
Checking Process P0
---------------------------------
Need of P0 : 7 4 3
Available   : 7 4 5
All required resources available
Process P0 CAN execute
Releasing resources of P0 back to system
Updated Available Resources: 7 5 5

---------------------------------
Checking Process P2
---------------------------------
Need of P2 : 6 0 0
Available   : 7 5 5
All required resources available
Process P2 CAN execute
Releasing resources of P2 back to system
Updated Available Resources: 10 5 7
Process P5 CANNOT execute now

SYSTEM IS NOT IN SAFE STATE
```

- **Conclusion:**

this practical, we conclude that the Banker's Algorithm effectively ensures safe resource allocation by preventing deadlocks. By calculating the current resource need each process and checking for a safe execution sequence, we can determine whether all processes can complete execution without system instability.

❖ Discussion Questions:

1. What is the purpose of the Banker's Algorithm?
Ans: The Banker's Algorithm is used for deadlock avoidance by ensuring safe resource allocation and checking if processes can execute without leading to system instability.

2. How do you calculate the current need of each process?
Ans: The current need of a process is calculated as:
Need = MAX - Allocation,
where **MAX** is the maximum required resources, and **Allocation** is the currently allocated resources.

3. What are the conditions for a system to be in a safe state?
Ans: A system is in a safe state if there exists a safe sequence of process execution where each process gets the required resources and completes execution without causing a deadlock.

4. What happens if no safe sequence exists in the Banker's Algorithm?
Ans: If no safe sequence exists, the system is in an unsafe state, meaning a deadlock could occur if resources are allocated improperly.

5. What are the key inputs required to implement the Banker's Algorithm?
Ans: The key inputs are:

- **Allocated Resources Matrix** (Resources currently allocated to each process)
- **Maximum Resources Matrix** (Maximum resources a process may need)
- **Available Resources Vector** (Resources currently available in the system)

❖ References:

https://www.geeksforgeeks.org/bankers-algorithm-in-operating-system-2/

https://chatgpt.com/

Date:____/____/2026

Signature
Course Coordinator
B.Tech CSE(AIML)
Sem: 4 / 2025-26